

Lecture 9: Target Code Generation

中文学习笔记

基于课件 9.Target_Code_Generation.pdf 整理

2026 年 6 月 27 日

目录

阅读导引	3
1 编译阶段与目标代码生成任务 [p1–6]	3
1.1 目标代码生成的位置 [p1–3]	3
1.2 三个主要任务 [p4–6]	3
2 栈式计算机、累加器与 MIPS 基础 [p7–13]	4
2.1 栈式计算机 [p7]	4
2.2 用累加器优化栈式机器 [p8–9]	5
2.3 从栈式机器到 MIPS [p10–13]	5
3 寄存器保存恢复、栈操作和表达式代码生成 [p14–20]	6
3.1 为什么要保存/恢复寄存器 [p14–15]	6
3.2 栈操作 [p16–18]	6
3.3 MIPS 翻译例子: $7 + 5$ [p17]	6
3.4 表达式代码生成策略 [p19]	7
3.5 条件语句代码生成 [p20]	8
4 运行时内存布局、活动记录和调用约定 [p21–30]	8
4.1 运行时内存区域 [p21]	8
4.2 Activation 和 Activation Record [p22–24]	9
4.3 Caller/Callee Conventions [p25–27]	10
4.4 详细调用步骤 [p28–30]	10

5 变量访问与帧指针偏移 [p31–33]	11
6 面向对象代码生成：对象布局与分派 [p34–39]	11
6.1 对象布局 [p34]	11
6.2 非虚方法、虚方法与分派 [p35–36]	12
6.3 典型对象布局与继承不变式 [p37–39]	12
7 机器相关优化 [p40–49]	13
7.1 机器相关优化总览 [p40]	13
7.2 指令选择与指令成本 [p41–43]	14
7.3 指令调度 [p44]	14
7.4 寄存器分配 [p45–46]	15
7.5 图着色与寄存器溢出 [p47–48]	16
7.6 窥孔优化 [p49]	16
8 总结 [p50]	17
9 练习题与解答	18
复习建议	20

阅读导引

本讲进入编译器后端的目标代码生成阶段。前面已经得到优化后的中间表示 IR；现在要把 IR 变成目标机器能执行的汇编或机器码。目标代码生成的核心矛盾是：IR 中临时变量很多、操作很抽象，而真实机器寄存器有限、指令集有具体限制、函数调用还必须遵守运行时栈和调用约定。

本讲主线

目标代码生成要解决三件事：选什么指令、哪些值放寄存器、指令按什么顺序执行。为了说明这些问题，课件先用栈式机器和 MIPS 建模，再讲运行时栈和调用约定，最后讨论对象分派和机器相关优化。

1 编译阶段与目标代码生成任务 [p1-6]

1.1 目标代码生成的位置 [p1-3]

目标代码生成（target code generation）位于优化后的中间代码和目标机器代码之间：

IR → Code Optimizer → Code Generator → target-machine code.

Target Code Generation

目标代码生成是把语法分析、语义分析和优化后得到的中间代码转换为目标代码的阶段。目标代码可以是汇编代码，也可以是机器码。

这一阶段要考虑：

- 如何让目标代码更短。
- 如何充分使用寄存器，减少内存访问。
- 如何利用目标机器指令系统的特点。

输入通常包括源程序 IR 和符号表；输出是目标程序。

1.2 三个主要任务 [p4-6]

任务	含义	难点
----	----	----

Instruction selection	选择合适的目标机器指令实现 IR 语句	同一 IR 操作可能有多种目标指令实现
Register allocation and assignment	决定哪些值保存在什么寄存器中	IR 临时变量近似无限，物理寄存器很少
Instruction ordering / scheduling	决定指令执行顺序	顺序影响寄存器需求、流水线等待和并行性

指令选择例子：

```
a = b + c;
d = a + e;
```

可翻译为：

```
LD  R0, b
ADD R0, R0, c
ST  a, R0
LD  R0, a
ADD R0, R0, e
ST  d, R0
```

如果后端能发现 a 的值仍在 $R0$ ，就可能避免重新 ‘LD R0, a’。这正是寄存器分配和指令调度会影响代码质量的原因。

寄存器分配和最优指令顺序选择在一般情况下很难，课件指出它们与 NP 问题相关；实践中通常使用启发式算法，而不是追求严格最优。

2 栈式计算机、累加器与 MIPS 基础 [p7-13]

2.1 栈式计算机 [p7]

Stack Machine

栈式计算机是一种简单求值模型：没有显式变量或寄存器，中间结果都放在一个值栈中。每条指令从栈顶取操作数、弹出操作数、完成运算，并把结果压回栈顶。

通俗理解：表达式求值像用一摞盘子做计算，最新产生的值总放在最上面，下一条指令从最上面拿操作数。

它的优点是模型简单；缺点是栈顶访问非常频繁，导致内存读写多。

2.2 用累加器优化栈式机器 [p8-9]

普通 ‘add’ 指令需要三次内存操作：读两个操作数，再写回一个结果。课件提出优化：把栈顶值保存在一个寄存器中，这个寄存器称为 accumulator（累加器）。

此时加法变成：

$$acc \leftarrow acc + top_of_stack.$$

这样只需要访问一次内存，即读取栈中另一个操作数。

对表达式 $3 + 7 + 5$ ，栈式求值会依次把操作数压栈/放入累加器，再做两次加法，最终结果保存在累加器中。

2.3 从栈式机器到 MIPS [p10-13]

课件用 MIPS 模拟带累加器的栈式机器：

- ACC 存在 MIPS 寄存器 \$t0。
- 栈存在内存中。
- \$sp 保存栈上下一可用位置地址。
- 栈顶位于 $\$sp + 4$ 。
- 栈向低地址增长，这是 MIPS 的标准约定。

MIPS

MIPS 是典型 RISC 架构。算术指令只对寄存器操作；若要使用内存中的值，必须先用 load 指令读入寄存器，计算后再用 store 写回内存。

MIPS 特点：

- 32 个 32-bit 通用寄存器。
- 本讲主要用 \$t0 作为 ACC，\$sp 作为栈指针，\$t1 作为临时寄存器。
- \$zero 恒为 0。

常见指令：

指令	含义
lw reg1 offset(reg2)	从地址 $reg2 + offset$ 读取 32-bit word 到 reg1
add reg1 reg2 reg3	$reg1 \leftarrow reg2 + reg3$

<code>sw reg1 offset(reg2)</code>	把 reg1 的 32-bit word 写到地址 <code>reg2+offset</code>
<code>addiu reg1 reg2 imm</code>	<code>reg1 <- reg2 + imm</code> , 不检查溢出
<code>li reg imm</code>	把立即数装入寄存器

3 寄存器保存恢复、栈操作和表达式代码生成 [p14-20]

3.1 为什么要保存/恢复寄存器 [p14-15]

IR 中可以有无限制临时变量，但真实机器寄存器有限，因此必须复用寄存器。复用带来问题：如果一个寄存器中旧值之后还要用，而我们要把新值写进去，就必须先保存旧值，之后再恢复。

例如生成 $E \rightarrow E_1 + E_2$ ：

- 先生成 E_1 ，结果在 `$t0`。
- 再生成 E_2 ，也会把结果写到 `$t0`，覆盖 E_1 的结果。
- 因此在生成 E_2 前，要把 E_1 的 `$t0` 保存到栈中。

保存 `$t0`：

```
sw    $t0, 0($sp)
addiu $sp, $sp, -4
```

恢复到 `$t0`：

```
lw    $t0, 4($sp)
addiu $sp, $sp, 4
```

这里栈向低地址增长；压栈后 `$sp` 下降，弹栈时 `$sp` 上升。

3.2 栈操作 [p16-18]

压入多个寄存器时，通常先移动 `$sp` 为新数据留空间，再保存：

```
addiu $sp, $sp, -8
sw    $t1, 4($sp)
sw    $t2, 0($sp)
```

弹出时只需要把 `$sp` 向高地址移动。被弹出的数据仍在内存里，但位于栈指针之外，按约定视为无效。

3.3 MIPS 翻译例子： $7 + 5$ [p17]

栈式机器代码：

```
acc <- 7
push acc
acc <- 5
acc <- acc + stack_top
pop
```

MIPS 翻译：

```
li    $t0, 7
sw    $t0, 0($sp)
addiu $sp, $sp, -4
li    $t0, 5
lw    $t1, 4($sp)
add   $t0, $t0, $t1
addiu $sp, $sp, 4
```

执行后，\$t0 中为 12，\$t1 中为 7，\$sp 回到压栈前位置。内存中原来保存 7 的位置可能仍有 7，但已经不属于有效栈内容。

3.4 表达式代码生成策略 [p19]

定义代码生成函数 $\text{cgen}(e)$ ：

- 生成表达式 e 的 MIPS 代码。
- 执行后 e 的值保存在 \$t0。
- 保持 \$sp 和栈内容一致。

对加法：

```
cgen(e1 + e2):
    cgen(e1)           # e1 result in $t0
    sw    $t0, 0($sp)
    addiu $sp, $sp, -4
    cgen(e2)           # e2 overwrites $t0
    lw    $t1, 4($sp)
    addiu $sp, $sp, 4
    add   $t0, $t1, $t0
```

如果只用 \$t1 暂存 e_1 ，对简单表达式可少用栈：

```
cgen(e1)
```

```

move $t1, $t0
cgen(e2)
add $t0, $t1, $t0

```

但若 e_2 自身也很复杂，例如 $3 + (7 + 5)$ ， $\$t1$ 可能被内部计算需要，仍需更系统的寄存器分配。

3.5 条件语句代码生成 [p20]

新增控制流指令：

- beq reg1 reg2 label: 若两个寄存器相等，跳到 label。
- b label: 无条件跳转。

对：

$$\text{if } e_1 == e_2 \text{ then } e_3 \text{ else } e_4$$

代码生成：

```

cgen(e1)
sw    $t0, 0($sp)
addiu $sp, $sp, -4
cgen(e2)
lw    $t1, 4($sp)
addiu $sp, $sp, 4
beq   $t0, $t1, true_branch
false_branch:
    cgen(e4)
    b end_if
true_branch:
    cgen(e3)
end_if:

```

基本思路是：比较前先保存 e_1 ，再计算 e_2 ，比较后选择 true/false 分支。

4 运行时内存布局、活动记录和调用约定 [p21-30]

4.1 运行时内存区域 [p21]

一个程序运行时常见内存布局：

区域	作用
----	----

Code	保存目标代码，生成后大小通常编译期确定
Global/static	保存全局常量、静态对象等编译期大小已知的数据
Heap	管理长生命周期动态数据，如对象和动态分配数组
Stack	保存动态过程调用信息，尤其是活动记录、局部变量、临时值

低地址通常放 code/data, heap 向高地址增长, stack 从高地址向低地址增长, 中间是 free memory。

4.2 Activation 和 Activation Record [p22-24]

Activation

一次过程或函数执行称为一个活动。活动从过程体开始执行时开始，过程结束后把控制权返回到调用点之后。

Activation Record

活动记录 AR 是管理某次过程执行所需信息的内存块，也称 stack frame 或 frame。调用栈中保存的就是一系列活动记录。

AR 栈管理：

- 函数入口：在栈顶分配新的 AR。
- 函数返回：从栈顶移除该 AR。
- \$sp 指向栈顶，移动 \$sp 即可分配/释放空间。
- \$fp 保存当前 frame 的基地址，局部变量和参数可用相对 \$fp 的偏移访问。

AR 内容通常包括：

- 临时变量。
- 局部变量。
- 保存的调用者/被调用者寄存器值。
- 调用者返回地址，也就是调用后下一条指令地址。
- 调用者的旧 \$fp。
- 参数。

- 返回值。

4.3 Caller/Callee Conventions [p25-27]

函数调用会覆盖寄存器。问题是：由调用者保存，还是被调用者保存？

- 调用者知道自己哪些寄存器值之后还要用。
- 被调用者知道自己会用哪些寄存器。
- 但二者不知道对方实现细节，因此需要约定。

课件给出合作方案：

类别	寄存器	责任
Caller-saved	\$t0-\$t9, \$a0-\$a3, \$v0-\$v1	被调用者可以修改；调用者若关心这些值，调用前自己保存，调用后恢复
Callee-saved	\$s0-\$s7, \$ra	调用者假设这些值调用后不变；被调用者若使用，必须保存和恢复

4.4 详细调用步骤 [p28-30]

调用者：

1. 为需要保存的 caller-saved 寄存器在栈上留空间并保存。
2. 参数从右到左逐个压栈。
3. 跳转到函数入口，并保存下一条指令地址到 \$ra。

被调用者：

1. 在栈上留空间，保存旧 \$fp 和 \$ra。
2. 设置自己的 \$fp 为当前 frame 的基地址。
3. 为局部变量和临时变量分配栈帧空间。
4. 执行函数体。

返回时，被调用者：

1. 将返回值放入 \$v0。
2. 弹出局部变量、临时变量、保存的寄存器。
3. 恢复 \$fp 和 \$ra。
4. 执行 jr \$ra 跳回调用者。

调用者恢复：

1. 弹出参数。
2. 恢复自己保存的 caller-saved 寄存器。

新增指令：

- jal label: 跳到函数入口，并把下一条指令地址保存到 \$ra。
- jr reg: 跳到寄存器中的地址，函数返回常用 jr \$ra。

5 变量访问与帧指针偏移 [p31-33]

函数的参数、局部变量和临时变量都在 AR 中。问题是：如果用 \$sp 相对地址访问变量，表达式求值期间不断压栈/弹栈会移动 \$sp，导致变量偏移不稳定。

解决方案是使用 \$fp：

- \$fp 在当前函数执行期间保持不动。
- 参数、局部变量和临时变量都可用固定 offset 访问。
- 参数通常在 \$fp 的正偏移处。
- 局部变量和临时变量通常在 \$fp 的负偏移处。

课件示例：

```
def f(x, y) = e
x: +8($fp)
y: +12($fp)
first local variable: -4($fp)
```

对嵌套调用，例如 fun1 调用 fun2，栈上会同时存在两个 AR；每个函数都有自己的 \$fp，因此相同 offset 在不同 AR 中指向不同函数的变量。

6 面向对象代码生成：对象布局与分派 [p34-39]

6.1 对象布局 [p34]

对象类似 C 结构体：

- 对象在内存中连续存放。
- 每个成员变量位于对象内固定 offset。
- 与结构体不同，对象还有成员方法。

成员访问可翻译为 offset 访问。例如：

```
obj.x = 10
```

可翻译为“把 10 写到 `obj + x_offset`”。

6.2 非虚方法、虚方法与分派 [p35-36]

Static Dispatch

静态分派是在编译期确定调用目标。非虚方法不能被覆盖，调用目标由静态引用类型决定，因此编译器可把函数地址硬编码到目标代码中。

Dynamic Dispatch

动态分派是在运行时根据对象真实类型选择调用目标。虚方法可被子类覆盖，因此同一个调用表达式在运行时可能跳到不同函数。

例子：

```
Parent obj = new Child();
obj.nonvirtual(); // 调用 Parent::nonvirtual()
obj.virtual();    // 调用 Child::virtual()
```

动态分派的实现方式：

- 编译期为每个类生成 dispatch table，表中保存该类所有虚方法的目标地址。
- 运行时每个对象带一个 dispatch pointer，指向其真实类的 dispatch table。
- 调用虚方法时，通过 dispatch pointer 加方法 offset 取出函数地址，再跳转调用。

6.3 典型对象布局与继承不变式 [p37-39]

典型对象布局：

- Class tag：用于动态类型检查。
- Dispatch ptr：指向 dispatch table。

- 成员变量，如 x, y, z 。

继承中必须保持一个不变式：

Offset Invariant

某个成员变量或成员方法在父类及其所有子类中的 offset 必须一致。这样通过父类引用访问对象时，即使对象真实类型是子类，编译器仍可使用父类布局的 offset。

例如：

- A1 有 x 、虚方法 $f1, f2$ 。
- A2 继承 A1，新增 y ，覆盖 $f2$ 。
- A3 继承 A2，新增 z ，新增 $f3$ 。

布局方式：

- 子类保留父类字段 offset，新增字段追加在后面。
- 子类覆盖虚方法时，dispatch table 中对应 offset 的函数地址替换成子类版本。
- 新增虚方法追加到 dispatch table 后面。

成员变量访问用引用类型对应的 offset；虚方法调用也用引用类型对应的 dispatch table offset。由于 offset invariant，真实对象是子类也能正确工作。

7 机器相关优化 [p40–49]

7.1 机器相关优化总览 [p40]

IR 优化后，编译器需要把优化后的 IR 转成目标语言。此时必须考虑具体架构特征：

- 专用指令。
- 硬件流水线能力。
- 寄存器数量和使用约定。

典型机器相关优化：

- 指令选择与调度。
- 寄存器分配。
- 窥孔优化。

7.2 指令选择与指令成本 [p41-43]

指令选择是在目标架构上为 IR 操作寻找高效汇编实现。它在 CISC 架构上尤其重要，因为一个 IR 操作可能有多种指令组合。

例如：

```
a = a + 1
```

可以实现为：

```
MOV a, R0
ADD #1, R0
MOV R0, a
```

也可以实现为：

```
INC a
```

后者更短、更便宜。

课件给出指令成本公式：

$$\text{Instruction cost} = 1 + \text{cost}(\text{source-mode}) + \text{cost}(\text{destination-mode}).$$

示例：

- $a := b + c$ 可用 `MOV b,R0; ADD c,R0; MOV R0,a`，成本约 6。
- 若允许直接内存目的的操作，可用 `MOV b,a; ADD c,a`，成本也可能约 6。
- 对间接地址 `*R1,*R0` 的操作成本可能更高。
- $a := a + 1$ 可从三条指令成本 6，降到 `ADD #1,a` 成本 3，再降到 `INC a` 成本 2。

7.3 指令调度 [p44]

Instruction Scheduling

指令调度是重排目标指令顺序以减少执行时间的优化。它利用硬件流水线和多发射能力，尽量减少等待周期、避免寄存器溢出并改善局部性。

例子：

```
A = x * y;  
B = A + 1;  
C = y;
```

若乘法延迟较长, 执行 $B = A + 1$ 必须等待 A 完成; 可改为:

```
A = x * y;  
C = y;  
B = A + 1;
```

这样 $C=y$ 可在等待乘法结果时执行, 减少空转。

7.4 寄存器分配 [p45-46]

Register Allocation

寄存器分配是把变量和表达式结果映射到有限物理寄存器, 并管理寄存器与内存之间数据转移的过程。

目标:

- 频繁访问的变量尽量放入寄存器。
- 变量只在其活跃期间占用寄存器。
- 减少内存 load/store, 因为内存访问代价高。

局部寄存器分配:

- 逐基本块分配。
- 决策简单, 但块之间映射可能不一致。

全局寄存器分配:

- 跨基本块做整体决策。
- 变量到寄存器的映射可跨块保持一致。
- 需要考虑 CFG 结构。

常见算法:

- 图着色分配器。

- 线性扫描分配器。
- 整数线性规划分配器。

7.5 图着色与寄存器溢出 [p47–48]

Register Interference Graph

寄存器干涉图 RIG 中，每个节点代表一个变量；若两个变量 live range 重叠，不能放入同一个寄存器，就在两个节点之间连边。

规则：

- 没有边的两个变量可共享同一寄存器。
- 有边的两个变量不能共享同一寄存器。
- k 个颜色对应 k 个寄存器。
- 若 RIG 可 k 着色，则程序可用 k 个寄存器分配。

Register Spilling

若寄存器不够，编译器把某些变量溢出到内存中。被溢出的变量不长期占用寄存器，需要使用时从内存 load，用完必要时 store 回内存。

k 着色判定是 NP-complete，因此 k 寄存器分配也是 NP-complete。实践中使用启发式多项式算法，例如 Chaitin 的图着色算法，GCC 后端中曾广泛使用类似思想。

溢出很慢，因为它引入额外内存读写。所以优秀的寄存器分配器对代码质量非常关键。

7.6 窥孔优化 [p49]

Peephole Optimization

窥孔优化是用一个小滑动窗口检查目标指令序列，并把窗口内指令替换为更短或更快序列的局部优化技术。

它通常有两种思路：

- 通过良好的指令选择和寄存器分配直接生成高质量代码。
- 先生成朴素目标代码，再用窥孔优化局部改进。

常见变换：

- 冗余指令消除。
- 控制流优化。
- 代数化简。

例子：jump to jump。

```
if a < b goto L1
...
L1: goto L2
```

可改成：

```
if a < b goto L2
...
L1: goto L2
```

条件跳转直接跳到最终目标，少执行一次无条件跳转。

8 总结 [p50]

代码可在不同层级优化：

- 窥孔优化、局部优化、循环优化、全局优化。
- IR 层优化：局部/全局、公共子表达式消除、常量折叠和常量传播。
- 目标层优化：指令选择、指令调度、寄存器分配等。

优化之间相互影响：

- 一个优化可能暴露另一个优化机会。
- 一个优化也可能隐藏或删除另一个优化机会。
- 最优优化组合仍是研究问题。
- 编译器通常把常一起使用的优化打包成等级，例如 -O1、-O2。

9 练习题与解答

题 1：目标代码生成的三个主要任务是什么？

解答：指令选择、寄存器分配与赋值、指令排序/调度。指令选择决定用哪些目标机器指令实现 IR；寄存器分配决定哪些值放入哪些寄存器；指令调度决定指令执行顺序，以减少等待并提高硬件利用率。

题 2：为什么栈式机器需要累加器优化？

解答：栈顶被频繁访问。普通栈式加法需要读两个操作数并写回结果，共三次内存访问。若把栈顶保存在累加器寄存器中，加法可变为 $acc \leftarrow acc + top_of_stack$ ，通常只需一次内存访问，减少开销。

题 3：执行课件中 $7 + 5$ 的 MIPS 代码后， $\$t0$ 、 $\$t1$ 和栈状态如何？

解答：执行后 $\$t0=12$ ，因为它保存 $5 + 7$ 的结果； $\$t1=7$ ，因为它从栈顶恢复了先前保存的 7； $\$sp$ 回到压入 7 之前的位置。原先保存 7 的内存单元可能仍有 7，但弹栈后不再属于有效栈内容。

题 4：写出 $e_1 + e_2$ 的基本代码生成策略。

解答：

```
cgen(e1)
sw    $t0, 0($sp)
addiu $sp, $sp, -4
cgen(e2)
lw    $t1, 4($sp)
addiu $sp, $sp, 4
add   $t0, $t1, $t0
```

先计算 e_1 ，保存到栈；再计算 e_2 ，恢复 e_1 到 $\$t1$ ，最后相加。

题 5：为什么访问函数变量要用 $\$fp$ 而不是 $\$sp$ ？

解答： $\$sp$ 会随着表达式求值、保存临时值、压栈/弹栈而变化，变量相对它的偏移不稳定。 $\$fp$ 在当前函数执行期间保持固定，因此参数和局部变量可以用固定偏移访问。

题 6：Caller-saved 和 Callee-saved 的区别是什么？

解答：Caller-saved 寄存器可被被调用者修改，调用者若需要保留它们的值，必须调用前保存、调用后恢复。Callee-saved 寄存器在调用后应保持不变，被调用者若使用它们，必须自己保存和恢复。

题 7：虚方法为什么需要动态分派？

解答：虚方法可被子类覆盖，调用目标取决于运行时对象真实类型，而不只是静态引用类型。因此编译期不能硬编码唯一目标，必须运行时通过对象的 dispatch pointer 查 dispatch table，找到真实类对应的函数地址。

题 8：继承中为什么要求成员变量和虚方法 offset 不变？

解答：父类引用可能指向子类对象。编译器根据引用类型生成固定 offset 的字段访问或虚方法表访问；若子类改变父类成员 offset，同一代码会访问错误位置。保持 offset 不变可保证父类代码对所有子类对象仍正确。

题 9：对 $a = a + 1$ ，为什么 INC a 可能比三条指令更好？

解答：三条指令实现需要 load、add、store，成本高且代码长。若目标机器支持 INC a，一条指令即可完成加一，通常更短、更快，指令成本也更低。

题 10：指令调度如何优化如下代码？

```
A = x * y;  
B = A + 1;  
C = y;
```

解答：因为 B 依赖 A ，而乘法可能有延迟，可把独立语句 $C = y$ 移到中间：

```
A = x * y;  
C = y;  
B = A + 1;
```

这样等待乘法结果时可以执行 $C = y$ ，减少空转。

题 11：寄存器干涉图中有边和无边分别表示什么？

解答：有边表示两个变量 live range 重叠，不能分配到同一寄存器；无边表示它们不会同时需要保留，可共享一个寄存器。k 可着色表示可用 k 个寄存器完成分配。

题 12：什么是寄存器溢出，为什么慢？

解答：当寄存器不够时，编译器把某些变量放到内存中，这叫 spilling。被溢出的变量使用前要从内存 load，用完后可能要 store 回内存。内存访问远慢于寄存器访问，因此溢出会显著降低性能。

题 13：什么是 jump to jump 窥孔优化？

解答：若条件跳转先跳到某标签，而该标签处又无条件跳到另一个标签，则可把条件跳转直接改到最终目标。例如：

```
if a < b goto L1
...
L1: goto L2
```

优化为：

```
if a < b goto L2
...
L1: goto L2
```

这样减少一次跳转。

复习建议

复习本讲建议抓住四条线：

- 代码生成线：IR 到目标代码，重点是指令选择、寄存器分配、指令调度。
- 栈和寄存器线：栈式机器、累加器、MIPS 栈指针、保存恢复、表达式代码生成。
- 运行时线：内存布局、活动记录、\$sp/\$fp、调用约定、函数调用。
- 后端优化线：指令成本、调度、寄存器干涉图、图着色、溢出、窥孔优化。

最容易混淆的是 \$sp 和 \$fp：\$sp 管理栈顶，随压栈弹栈移动；\$fp 固定当前活动记录基准，适合稳定访问参数和局部变量。